

INTERNSHIP REPORT

**SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS
FOR THE AWARD
OF**

**DIPLOMA
IN
COMPUTER ENGINEERING**

Submitted by:-

Pratik Antaram Lanjewar – 2213036

Under the supervision of

Mr. A.A. Uike

Lecturer

Department of Computer Engineering



DEPARTMENT OF COMPUTER ENGINEERING

GOVERNMENT POLYTECHNIC, NAGPUR

(An Autonomous Institute of Govt. of Maharashtra)

(Mangalwari Bazaar, Nagpur-440001)

2024-25

Informatrix IT Solutions Pvt Ltd.

**54, Central Bazar Road, nr. Vodafone Store, Bajaj Nagar,
Nagpur, Maharashtra 440010.**

+91 82379 31287

<https://www.informatrixIntd.com/LnTDApp/>

CERTIFICATE

GOVERNMENT POLYTECHNIC, NAGPUR

(An Autonomous Institute of Government of Maharashtra)



This is certified that,

Pratik Antaram Lanjewar- 2213036

Of term EVEN 2024 of Computer Engineering department student have submitted their Final Internship report during academic session 2024-2025 as a part of internship work prescribed by Government Polytechnic, Nagpur for the partial fulfillment of the requirement of Diploma in Computer Engineering, Sixth Semester. The internship work is the record of student's own work and is completely satisfactory.

Mr. A.A. Uike
(Mentor)

CERTIFICATE

GOVERNMENT POLYTECHNIC, NAGPUR

(An Autonomous Institute of Government of Maharashtra)



This is certified that,

Pratik Antaram Lanjewar – 2213036

Of term EVEN 2024 of Computer Engineering department student have submitted their Final Internship report during academic session 2024-2025 as a part of internship work prescribed by Government Polytechnic, Nagpur for the partial fulfillment of the requirement of Diploma in Computer Engineering, Sixth Semester. The internship work is the record of student's own work and is completely satisfactory.

**INTERNAL
EXAMINER**

**EXTERNAL
EXAMINER**

DATE:

GOVERNMENT POLYTECHNIC, NAGPUR

(Mangalwari Bazaar, Nagpur – 440001)

DEPARTMENT OF COMPUTER ENGINEERING

CANDIDATE'S DECLARATION

I, student of Diploma (Computer Engineering Department), hereby declare that the Internship Report which is submitted by us to the Department of Computer Science & Engineering, Government Polytechnic, Nagpur in sixth semester for internship which is part of diploma course. Projects created during this period is purely self-made with full of courage and enthusiasm some other skills are also included. The project encompasses a DevOps-based Automated Server Scaling & Load Management, designed with a clear separation of presentation, logic, and data layers. This application forms the core of my internship experience at Informatrix and is comprehensively detailed in the subsequent report.

Place: Nagpur

Date:

Name	Enrollment no.	Sign
Pratik Antaram Lanjewar	2213036	

This is to certify that the above statement made by the candidate is correct to best of our knowledge.

Mr. A.A. Uike
(Mentor)

ACKNOWLEDGEMENT

The success of this internship owes much to the assistance and input of numerous individuals within the organization. We extend our heartfelt gratitude to **Mr. A.A. Uike**, whose guidance and support provided us with the invaluable opportunity to undertake these projects during our internship. His unwavering encouragement and belief in the importance of the learning process greatly influenced our journey. We are immensely thankful to all the faculties for their constant guidance, monitoring, and motivation throughout the progress evaluations. Their insights, suggestions, and unwavering support played a pivotal role in shaping the outcome of all the projects. Their dedication to fostering our growth, sharing new ideas, and providing essential information were instrumental in our journey, with the exchange of view and under his guidance we accomplish our goal projects.

We express our sincere gratitude to **Dr. M. V. Sarode**, Head of Computer Engineering for his stimulating guidance, continuous encouragement and supervision throughout the course of present work. Under his guidelines we come to know about this internship schedule and concluded our internship on time with precision. We would again like to thank **Dr. R.P Mogre**, Principal of our College, for providing us the necessary help and facilities we needed.

TABLE OF CONTENTS

<i>Certificate</i>	<i>i</i>
<i>Certificate for External</i>	<i>ii</i>
<i>Candidate's Declaration</i>	<i>iii</i>
<i>Acknowledgement</i>	<i>iv</i>

TITLE	PAGE NO.
Chapter 1: INTRODUCTION	1
1.1 Organization Overview	2
1.1.1 Mission and Values	2
1.1.2 Activities	2
1.1.3 Clientele	3
1.1.4 Collaborative Culture	3
1.1.5 Commitment to Quality	3
1.2 Internship Duration & Timeline	3
1.2.1 Timeline Overview	3
1.3 Report Overview	4
Chapter 2: OBJECTIVES OF THE INTERNSHIP	6
2.1 Personal Objective	7
2.1.1 Skill Differentiation	7
2.1.2 Soft Skill	7
2.1.3 Project Engagement	8
2.2 Alignment with Academic and Career Goals	9
Chapter 3: DESCRIPTION OF THE ORGANIZATION	10
3.1 History and Structure	11
3.2 Projects	11
3.3 Target Market/Clients	12
Chapter 4: DEPARTMENT OR DIVISION DETAILS	13
4.1 Placement Details	14
4.2 Roles and Responsibilities of Department	14

4.3 Key Functions of the Department	14
-------------------------------------	----

Chapter 5: INTERNSHIP ACTIVITIES	16
---	----

5.1 Project Details	17
5.2 Project Objectives	17
5.3 Methodology	18
5.4 Output	29
5.5 Challenges Faced and Overcoming Them	34
5.6 New Skills and Knowledge Gained	34

Chapter 6: ACHIVEMENTS AND CONTRIBUTIONS	39
---	----

6.1 Significant Accomplishments	37
6.2 Contributions and Organizational Benefits	38
6.3 Positive Feedback and Recognition	38

Chapter 7: LEARNING OUTCOMES	39
-------------------------------------	----

7.1 Learning Experience	40
7.2 Relating Experiences to Academic Coursework	40
7.3 Impact on Career Goals	41

Chapter 8: CHALLENGES AND LESSON LEARNED	42
---	----

8.1 Encountered Challenges and Solutions	43
8.2 Areas for Improvement in Project	43
8.3 Lessons Learned	44

Chapter 9: RECOMMENDATIONS	45
-----------------------------------	----

9.1 Feedback for Organization	46
9.2 Areas of Improvement	46
9.3 Recommendations for Future Interns	46

Chapter 10: CONCLUSION	48
-------------------------------	----

10.1 Summary	49
10.2 Gratitude	49

CHAPTER 1

INTRODUCTION

1.1 Organization Overview

Informatrix IT Solutions Private Limited , Founded in 2020 , is a leading IT Infrastructure Management Outsourcing firm based in India. Committed to closing the gap between technology and business operations, the company excels in executing IT infrastructure projects and managing ongoing operations. Informatrix delivers comprehensive consulting services tailored to the evolving demands of modern enterprises. The company also runs a specialized training center, equipping over 600 IT professionals each year with expertise in emerging technologies such as Linux, DevOps, Cloud Computing, and various Open Source Tools.

1.1.1 Mission and Values

At Informatrix, our goal is to empower organizations with forward-thinking, dependable, and scalable IT solutions that drive digital transformation and boost operational performance. Our actions are guided by a strong set of core principles—creativity, honesty, client-centricity, excellence, and teamwork.

These principles are deeply rooted in our company culture and shape every service, interaction, and decision we make. We are dedicated to maintaining ethical standards, nurturing lasting client partnerships, delivering top-notch quality, and promoting a mindset of constant growth and development. Fueled by collaboration and a love for technology, we aim to be a reliable ally in our clients' journey to success.

1.1.2 Activities

Informatrix has delivered several impactful projects that have become landmarks, strengthening its identity in the IT services sector. Key highlighted projects include:

- **Nagios Monitoring Systems:** Deployed Nagios for real-time IT infrastructure monitoring to maintain system availability.
- **Private GitLab:** Implemented a self-hosted GitLab instance to manage source control and enhance DevOps collaboration workflows.
- **User Authentication via GCPW:** Integrated Google Credential Provider for Windows (GCPW) to offer secure single sign-on (SSO) for enterprise devices.
- **ERP with Salesforce:** Streamlined core business functions such as HR, sales, and inventory using the Salesforce ERP platform.
- **Linux Server Security Using Fail2ban:** Configured Fail2ban for intrusion prevention, shielding Linux servers from brute-force login attempts.

1.1.3 Clientele

- **SquareOne DataLabs**
Specialists in advanced data analytics and delivering intelligent, data-driven solutions.
- **Beedu Services Pvt. Ltd.**
Providers of comprehensive IT solutions and custom software development services.
- **BANKiQ**
Experts in fraud detection, risk management, and compliance technologies.

1.1.4 Collaborative Culture

At Informatrix IT Solutions, a strong collaborative culture forms the foundation of its success. The company promotes an environment of teamwork, continuous learning, and innovation, encouraging open communication and shared goals across all levels. With a customer-first mindset, Informatrix emphasizes proactive IT management, guided by principles of integrity, transparency, and accountability in every interaction.

The firm prioritizes talent enhancement by offering ongoing professional development and upskilling programs. It also ensures a healthy work-life balance and a nurturing workplace ethos. Informatrix remains devoted to security, compliance, and high standards, ensuring all solutions are delivered with reliability and trust.

1.1.5 Commitment to Quality

Informatrix IT Solutions maintains a steadfast commitment to quality across its diverse and innovative service offerings. By applying proven methodologies, advanced technologies, rigorous testing protocols, and continuous assessment strategies, the company consistently aims to surpass evolving client expectations. Its unwavering dedication to quality is evident in each project, executed with accuracy, dependability, meticulous attention to detail, and a clear vision for long-term achievement and sustainability.

1.2 Internship Duration & Timeline

- **Duration:** 4 months or 16 weeks
- **Start Date:** 13th January 2025
- **End Date:** 30th April 2025

1.2.1 Timeline Overview:

Weeks 1-5:

- **Week 1: Introduction to Informatrix and Linux Fundamentals**
Got introduced to Informatrix's ecosystem and Linux basics. Installed Rocky Linux using VirtualBox to build familiarity with the Linux interface.
- **Week 2: Basic Linux Commands**
Learned essential Linux commands for managing files, users, and networks. These foundational skills enabled efficient system navigation and basic troubleshooting techniques.
- **Week 3: Bash Scripting**
Explored Bash scripting concepts like variables, loops, and conditions. Learned user and process management for automation and simplified routine administrative tasks.
- **Week 4: Advanced Linux Topics**
Dived deeper into Linux with sudo privileges, vi editor, LVM configuration, cron jobs, and package management for advanced system-level operations and task scheduling.
- **Week 5: DevOps Fundamentals and Version Control**
Understood DevOps principles, software development lifecycle, and importance of CI/CD. Practiced version control using Git and GitHub for collaborative and reliable development.

Weeks 6-10:

- **Week 6: Docker Basics**
Installed Docker, understood containers vs. VMs, and learned core Docker concepts. Built, ran, and managed containers to isolate applications and simulate microservice-like environments in a lightweight, consistent way.
- **Week 7: Docker Compose & Dockerfile**
Built Dockerfiles to create customized images for applications. Used Docker Compose to manage and orchestrate multi-container apps in structured projects.
- **Week 8: Jenkins & CI/CD**
Installed Jenkins and explored pipeline as code. Configured automated builds using freestyle projects and scripted pipelines. Connected GitHub repos and Docker images to streamline the CI/CD process for efficient software deployment.
- **Week 9: Ansible Automation**
Installed and configured Ansible for infrastructure automation. Wrote and executed playbooks to manage users, install software, and perform system configuration tasks across multiple remote servers simultaneously using SSH.

- **Week 10: AWS & Cloud Basics**

Created and explored an AWS account using the console and CLI. Launched and secured EC2 instances. Connected remotely, installed applications, and learned basic cloud infrastructure principles like scalability and pay-per-use.

Weeks 11-16

- **Week 11 – Project Planning & Tool Selection**

Choose the necessary tools for automating server scaling and load management: Python Flask (for the web application), Nginx (for reverse proxy and load balancing), Terraform (for infrastructure management), and Ansible (for automated configuration). Set up the basic environment for Flask, Nginx, Terraform, and Ansible. Implemented a simple Flask app for testing purposes and created an initial Terraform script to provision an EC2 instance. Learned Nginx, Terraform, Ansible independently

- **Week 12 – Dockerization & Instance Setup**

Dockerized the Flask app, deployed EC2 instances using Terraform, and automated instance setup with Ansible to install Docker and configure the Flask app.

- **Week 13 – Implementing Server Scaling Logic**

Implemented scaling logic based on CPU usage and active connections, integrated Terraform to provision new instances as required based on traffic.

- **Week 14 – Traffic Management with Nginx**

Configured Nginx for dynamic load balancing, ensuring new instances were automatically added to the load balancing pool after each scaling operation.

- **Week 15 – Automation with Terraform and Ansible**

Automated scaling and deployment using Terraform and Ansible, ensuring new instances were provisioned, configured, and added to the Flask app setup automatically.

- **Week 16 – Testing, Validation & Final Documentation**

Tested the system's scalability under heavy traffic, validated automated scaling, and documented the entire setup and integration process for future reference.

1.3 Report Overview

This report summarizes our journey, highlighting the goals set, challenges faced, and skills developed. It reflects our growth, showcasing how we overcame obstacles using our skills and others' insights. It also emphasizes improvements in presentation and practical skills, which helped us focus on future objectives.

CHAPTER 2

OBJECTIVES OF THE INTERNSHIP

2.1 Personal Objectives:

2.1.1 Skill Differentiation:

- **Expertise in Programming Languages:**

In this project, I applied my backend development skills using Python with the Flask framework to build a lightweight and scalable web application. For the frontend, I ensured a simple and responsive UI using basic HTML, making the application user-friendly and accessible.

- **DevOps Skills:**

I implemented Docker for containerizing the Flask application and used Ansible for configuration management. Terraform was used for infrastructure provisioning, while the entire process was automated to support dynamic server scaling. This project strengthened my understanding of DevOps practices and deployment pipelines.

2.1.2 Soft Skill:

- **Ability to solve problems:**

During the project, I faced challenges in integrating backend logic for dynamic scaling and configuring infrastructure automation. I overcame these issues by learning and applying tools like Terraform for provisioning and Ansible for server setup, which streamlined deployment and resource management.

- **Enhanced Presentation Abilities:**

Throughout the development process, I presented key design decisions, scaling logic, and automation flow to team members and mentors. This helped me clearly articulate technical concepts, improving my confidence and communication skills when addressing both technical and non-technical audiences.

- **Adaptability to New Technologies:**

This project introduced me to several new technologies such as Terraform, Ansible, AWS EC2, and Nginx load balancing. Adapting quickly, I gained hands-on experience and effectively implemented these tools in a real-world scenario, showcasing my ability to learn and apply emerging technologies in a fast-paced environment.

2.1.3 Project Engagement:

PROJECT: "Automated Server Scaling & Load Management of Dockerized Flask Application using Terraform , Ansible , Nginx and AWS"

- **Planning:**

In this project focused on dynamic server scaling and load management, I began with meticulous planning to ensure high availability, zero downtime during scaling, and seamless integration of new instances. I designed the system so that one set of Flask application instances would handle live traffic, while others could be prepared in advance for upgrades or scaling. Using Nginx as a reverse proxy, traffic could be rerouted dynamically to new or scaled instances. This setup ensured flexibility, fault tolerance, and quick recovery by automatically provisioning or replacing instances with minimal impact on users. The use of Terraform and Ansible allowed me to roll out changes in a controlled, repeatable way.

- **Requirements:**

The main technologies used in this project included Flask for backend logic and HTML/CSS for the frontend. Infrastructure was provisioned automatically on AWS EC2 using Terraform, while Ansible handled server setup and deployment. The Flask app was containerized with Docker for consistent deployment. Nginx acted as a reverse proxy and load balancer to manage incoming traffic. Performance monitoring was enabled via mpstat and the Nginx status module, which helped detect load spikes and trigger scaling. The setup ran on Ubuntu-based EC2 instances (minimum 4 GB RAM), with GitHub used for version control and integration with Jenkins or manual deployment triggers.

- **Setup:**

The Flask app was first built and containerized using Docker, with the image pushed to Docker Hub. Terraform scripts were written to provision EC2 instances, configure networking (VPCs, subnets, security groups), and output IPs. These IPs were passed to Ansible, which installed Docker, pulled the container image, and started the Flask app. Nginx was configured to distribute traffic across running instances by dynamically updating its upstream list. This automation ensured new instances could be integrated smoothly during scaling, maintaining performance without service disruption.

2.2 Alignment with Academic and Career Goals:

Objective	Academic	Careers
Theoretical knowledge application.	Apply theoretical knowledge to enhance understanding and academic output.	Use knowledge for project reports and client presentations.
Soft Skills	Develop diligence and improve learning ability.	Strengthen teamwork and leadership skills.
Networking	Engage with peers to exchange ideas and gain insights into their experiences.	Collaborate with senior professionals and clients for better project outcomes.
Goal Setting	Set academic targets for exams and assignment submissions.	Establish project deadlines and seek timely feedback for improvement.

CHAPTER 3

DESCRIPTION OF THE ORGANIZATION

3.1 History and Structure:

Informatrix IT Solutions was established in 2020 with a vision to close the technological gaps in IT infrastructure management. Since its inception, the company has grown rapidly by delivering an extensive array of services such as cloud migration, cybersecurity management, DevOps automation, and 24/7 infrastructure support. Its core mission is to assist organizations in streamlining their IT operations and boosting overall efficiency through tailored technological solutions.

The organization's structure is carefully designed to address a wide spectrum of IT requirements. While the leadership team defines strategic initiatives, dedicated teams manage specialized domains including IT infrastructure, DevOps, cloud transitions, and project execution. Informatrix places strong emphasis on professional development, conducting an annual training initiative that empowers over 600 individuals with up-to-date industry competencies. As part of its forward-looking growth plan, the company intends to expand its team to over 100 members by December 2024, reinforcing its impact and footprint in the IT services sector.

3.2 Projects:

Software Flexibility:

Informatrix IT Solutions delivers customizable software solutions tailored to the unique requirements of each business. This adaptability ensures smooth integration with existing systems, allowing companies to maintain their current workflows without major adjustments. Key implementations that reflect this flexibility include:

- **Nagios Monitoring Systems:** Deployed Nagios to actively oversee IT infrastructure, ensuring consistent system availability.
- **Private GitLab:** Deployed an in-house GitLab server for secure source code management and improved DevOps workflow.
- **User Authentication Using GCPW:** Employed Google Credential Provider for Windows (GCPW) to offer secure single sign-on (SSO) functionality across enterprise devices.
- **Enable MFA-Delete on S3 Bucket:** Strengthened AWS security by enabling multi-factor authentication (MFA) for deleting items in S3 buckets.

Consulting Services:

Informatrix delivers focused IT consulting services to assist businesses in identifying inefficiencies in their infrastructure and offering custom-fit solutions. These services encompass cloud transitions, security enhancements, system consolidation, and streamlining processes—all with the goal of increasing efficiency and supporting business growth.

Training Programs:

Informatrix strongly believes in continuous skill advancement and professional growth. It offers in-depth training initiatives that ensure professionals remain knowledgeable about the latest tech trends. These programs emphasize vital domains such as DevOps, Linux, IT security, cloud platforms, and open-source technologies—ensuring businesses can rely on proficient personnel to solve advanced IT challenges.

3.3 Target Market/Clients:

- **Data Analytics and IT Service Providers** – For instance, SquareOne DataLabs and Beedu Services Pvt. Ltd., deliver comprehensive data-driven insights and a broad spectrum of innovative technology services tailored to business needs.
- **Financial, Compliance, and Healthcare Sectors** – Companies such as BANKiQ focus on advanced fraud detection, effective risk management strategies, and secure IT solutions specifically designed for healthcare systems and sensitive data protection.
- **Enterprises Seeking Custom Software and Consulting** – Organizations looking for highly customized software applications and expert IT consulting support, including infrastructure evaluations, cloud transformation services, and enhanced cybersecurity measures.
- **Retail, E-commerce, and Manufacturing** – Firms like RetailTech Innovations and SupplyChain Solutions offer cutting-edge IT solutions to streamline retail processes, automate inventory systems, and optimize supply chain workflows efficiently.
- **Companies Investing in Workforce Upskilling and Education** – Businesses that prioritize continuous professional development through training programs in DevOps, cloud computing, Linux, cybersecurity, and modern educational technology platforms.

CHAPTER 4

DEPARTMENT OR DIVISION DETAILS

4.1 Placement Details:

Securing an internship in the field of computer science was both a refreshing and rewarding experience. I had the opportunity to explore professional IT practices at Informatrix IT Solutions Pvt. Ltd., which opened new avenues for learning and skill development. Initially, the transition into the professional domain posed a few challenges, but the company's structured training program and continuous support system helped me adapt quickly. During the internship, I worked on a project titled "Automated Server Scaling and Load Management using Terraform, Ansible, and NGINX", where I gained practical knowledge in infrastructure automation, configuration management, and efficient traffic distribution—key components of scalable cloud architectures.

4.2 Roles and Responsibilities of Department

The IT Infrastructure & Cloud Services Department at Informatrix IT Solutions is vital to the design, implementation, and optimization of secure and scalable IT infrastructures for a wide range of industries. Their primary responsibility involves the integration and orchestration of cloud services such as AWS and Azure, along with emerging technologies like infrastructure automation tools. The department aims to maximize performance and availability through automated resource provisioning using Terraform, system configuration via Ansible, and traffic management with NGINX. A major focus is also on implementing robust DevOps pipelines that improve deployment cycles and system reliability while managing cloud resources dynamically to meet changing application loads.

4.3 Key Functions of the Department:

- **Specification:**

The department's key objective is to deliver adaptable, high-performance, and secure infrastructure tailored to each client's business needs. In the context of automated server scaling and load management, the department handles dynamic provisioning of virtual resources and the intelligent distribution of network traffic. Using tools like Terraform for IaC and Ansible for configuration automation, the team ensures that infrastructure changes are fast, reliable, and reproducible. Load balancing with NGINX helps maintain high availability and user responsiveness even under fluctuating demand, minimizing risks during scaling events or deployments.

- **Technology Stack:**

The project makes use of Terraform to define and provision infrastructure across cloud

platforms in a declarative way. Ansible is used for automated configuration and deployment tasks, reducing manual errors and speeding up environment setup. NGINX, acting as a reverse proxy and load balancer, ensures efficient routing of client requests to the optimal backend instances. Together, these technologies automate the entire deployment pipeline and scale resources up or down based on real-time demand, ensuring reliability and system resilience.

- **User Experience:**

With automated server scaling and intelligent load management, the user experience is significantly enhanced. Applications remain responsive and accessible even during peak traffic periods or system updates. The use of NGINX as a load balancer ensures even distribution of traffic across active servers, preventing system overload and ensuring smooth operation. Automated configuration and scaling enable faster recovery and better performance, resulting in a highly available and stable user-facing application.

- **Future Scope:**

Future enhancements of this project include deeper integration with serverless computing models to reduce infrastructure costs and management overhead. Additional improvements may involve implementing auto-healing mechanisms using cloud-native monitoring tools, and incorporating AI-based predictive scaling to respond to traffic patterns proactively. Expanding the current setup to support multi-cloud deployments is also a promising avenue to ensure higher redundancy, performance optimization, and business continuity.

CHAPTER 5

INTERNSHIP ACTIVITIES

5.1 Project Details:

The title of the project is "**Automated Server Scaling and Load Management using Terraform, Ansible, and NGINX**". This project falls within the domain of Cloud Infrastructure Automation and DevOps, with a focus on dynamic scaling, configuration management, and load balancing of services based on real-time usage metrics. It provides a self-healing and scalable server setup by integrating infrastructure provisioning and configuration tools, all orchestrated from a centralized control node.

The key technologies used are:

- **Terraform:** An Infrastructure as Code (IaC) tool used to provision and manage AWS EC2 instances based on traffic and load.
- **Ansible:** An automation engine used to configure new EC2 instances post-deployment (install Docker, configure services).
- **Docker:** A container platform used to standardize application deployment across all nodes.
- **NGINX:** Acts as a reverse proxy and load balancer, dynamically updated to route traffic to available backend instances.
- **AWS EC2:** Provides scalable computing resources for hosting the main server and additional nodes.
- **AWS CLI:** Enables secure, scriptable access to AWS services from the control node.
- **Bash Scripting:** Automates monitoring of system metrics (CPU, active users) and triggers scaling logic.

The core idea is to keep one main EC2 instance always running and monitor its performance. When system load exceeds set thresholds (e.g., CPU > 80%, active connections > 300), a new EC2 instance is automatically created using Terraform, configured with Ansible, and added to the NGINX load balancer—ensuring uninterrupted service delivery.

5.2 Project Objectives

The objective of this project is to create an automated infrastructure scaling system that dynamically provisions, configures, and integrates new server instances into a load-balanced architecture using Terraform, Ansible, Docker, and AWS, all managed from a central EC2 node.

Key objectives include:

1. **Real-Time Monitoring and Trigger-Based Scaling:**

Continuously monitor CPU usage and active NGINX connections. Automatically trigger scaling operations when predefined thresholds are exceeded.

2. **Automated Infrastructure Provisioning with Terraform:**

Use Terraform scripts to create new EC2 instances programmatically, ensuring scalability and repeatability.

3. **Seamless Server Configuration with Ansible:**

Once an instance is created, use Ansible playbooks to install and configure Docker and necessary application dependencies, enabling zero manual setup.

4. **Dynamic Load Balancing via NGINX:**

Automatically update the NGINX upstream configuration to route traffic to new instances. Reload NGINX configuration with no downtime to accommodate new nodes.

5. **Infrastructure State Management:**

Maintain and update a local count of active instances to track system scale and assist in determining future provisioning needs.

6. **Single Control Node Management:**

Centralize all operations—monitoring, scaling, configuration, and load balancing—on a single EC2 instance for streamlined orchestration.

5.3 Methodology

1. **Infrastructure Setup**

The infrastructure for this project is built on **Amazon Web Services (AWS)** using **Terraform** for provisioning and managing the resources, and **Ansible** for configuring the software and ensuring consistency across the servers. The main server is responsible for monitoring CPU usage and managing traffic across multiple worker nodes using Nginx as a reverse proxy.

- **AWS EC2 Instances:** The infrastructure consists of a main server and dynamic worker nodes (EC2 instances), which are launched as needed based on resource utilization.
- **Security Setup:** A security group is configured to allow SSH, HTTP, HTTPS, and application traffic on port 5000, ensuring that the instances are accessible for management and communication.

2. Dockerization of Flask App

- **Docker ensures application portability and consistency**
 - Docker plays a fundamental role in the **Automated Server Scaling** project by packaging the Flask application and all its dependencies into a lightweight container.
 - This ensures that the application runs consistently across all development, testing, and production environments.
- **Containerization using python:3.10-slim base image**
 - The Flask application is containerized using the python:3.10-slim base image, which provides a minimal Python environment with reduced image size for faster builds and deployments.
- **Simplified deployment process**
 - The application is served using Flask and listens on port 5000.

❖ DockerFile :-

```
FROM python:3.10-slim
WORKDIR /app
COPY . .
RUN pip install Flask==2.3.2
EXPOSE 5000
CMD ["python", "app.py"]
```

3. Monitoring and Scaling Script

A key component of the project is a **Bash script** running on the main server, responsible for:

- **Monitoring CPU Utilization:** The script uses the mpstat command to monitor the server's CPU usage. If the CPU usage exceeds a defined threshold (e.g., 80%), the system will consider scaling.
- **Monitoring Active User Connections:** The script fetches the number of active connections to the application via Nginx's stub_status page. Based on this information, it calculates the number of required EC2 instances.
- **Triggering Scaling:** If both CPU usage and active connections exceed defined thresholds, the script will initiate the process of scaling up by provisioning new EC2 instances using Terraform. Once new instances are created, the script ensures that they are configured with **Docker** (to run the Flask application) and **Nginx** (to handle load balancing).

- **Instance Scaling Logic:** The script tracks the number of active instances and ensures that the necessary number of worker nodes is created based on the workload. It uses the Terraform apply command to launch new instances, and the Ansible playbook to configure them with Docker, the application, and Nginx.

❖ **Increment_instance.sh :-**

```
#!/bin/bash

# Configuration paths
NGINX_UPSTREAM_FILE="/etc/nginx/upstreams/new_ips.conf"
INSTANCE_COUNT_FILE="instance_count.txt"
TERRAFORM_DIR="/home/ubuntu/terraform"
ANSIBLE_PLAYBOOK="/home/ubuntu/terraform/configure_docker.yml"
ANSIBLE_HOSTS="/etc/ansible/hosts"
SSH_KEY="$TERRAFORM_DIR/projectkey"
SSH_USER="ubuntu"
SCALING_IN_PROGRESS=0

# Initialize instance count if not exists
if [ ! -f "$INSTANCE_COUNT_FILE" ]; then
    echo "0" > "$INSTANCE_COUNT_FILE" # Initially no extra instances
fi

# Main loop
while true; do
    if [[ "$SCALING_IN_PROGRESS" -eq 1 ]]; then
        echo "Scaling already in progress, waiting..."
        sleep 10
        continue
    fi

    # Get CPU usage
    CPU_USAGE=$(mpstat 1 1 | awk '/^Average:/ {print 100 - $NF}')

    # Get active Nginx connections
    ACTIVE_CONN=$(curl -s http://127.0.0.1/nginx_status | grep 'Active' | awk '{print $3}')

    # Calculate required number of instances based on traffic
    REQUIRED_INSTANCES=$(echo "scale=0; (($ACTIVE_CONN - 1)/250) + 1" | bc)

    BASE_INSTANCES=1 # Main server always active
```

```

ADDITIONAL_INSTANCES=$(cat "$INSTANCE_COUNT_FILE")
CURRENT_INSTANCES=$((BASE_INSTANCES + ADDITIONAL_INSTANCES))
echo "CPU Usage: $CPU_USAGE%"
echo "Active Connections: $ACTIVE_CONN"
echo "Required Instances: $REQUIRED_INSTANCES"
echo "Current Instances: $CURRENT_INSTANCES"
CPU_HIGH=$(echo "$CPU_USAGE > 80" | bc)
if [[ "$CPU_HIGH" -eq 1 && "$ACTIVE_CONN" -gt 300 && "$CURRENT_INSTANCES" -
lt "$REQUIRED_INSTANCES" ]]; then
    echo "High load detected. Scaling up..."
    SCALING_IN_PROGRESS=1
    NEW_COUNT=$((ADDITIONAL_INSTANCES + 1))
    # Run Terraform to launch new instance
    terraform apply -var="instance_count=$((NEW_COUNT))" -auto-approve
    # Get new IP from Terraform output
    NEW_IP=$(terraform -chdir="$TERRAFORM_DIR" output -raw latest_instance_ip
2>/dev/null)
    if [ -n "$NEW_IP" ]; then
        echo "New instance IP: $NEW_IP"
        # Wait for SSH
        until ssh -o StrictHostKeyChecking=no -i "$SSH_KEY" "$SSH_USER@$NEW_IP" "echo
'SSH Ready'" &>/dev/null; do
            echo "Waiting for SSH on $NEW_IP..."
            sleep 10
        done
        # Add to Ansible hosts
        echo "server_$NEW_COUNT ansible_host=$NEW_IP" | sudo tee -a
"$ANSIBLE_HOSTS" > /dev/null
        # Wait for cloud-init to finish
        ssh -o StrictHostKeyChecking=no -i "$SSH_KEY" "$SSH_USER@$NEW_IP" "cloud-init
status --wait"
        # Run Ansible playbook
        ansible-playbook -i "$ANSIBLE_HOSTS" "$ANSIBLE_PLAYBOOK"
        # Update Nginx upstreams

```

```

sudo sed -i '$d' "$NGINX_UPSTREAM_FILE"
echo "    server $NEW_IP:5000 ;" | sudo tee -a "$NGINX_UPSTREAM_FILE" > /dev/null
echo "}" | sudo tee -a "$NGINX_UPSTREAM_FILE" > /dev/null

# Reload Nginx
sudo nginx -t && sudo systemctl reload nginx

# Update instance count
echo "$NEW_COUNT" > "$INSTANCE_COUNT_FILE"
else
    echo "ERROR: Could not get new instance IP!"
fi

SCALING_IN_PROGRESS=0
fi

sleep 3
done

```

4. Terraform for Infrastructure Provisioning

Terraform is used to provision EC2 instances and configure them based on the infrastructure requirements. This includes creating:

- **EC2 Instances:** The EC2 instances are provisioned with an **Ubuntu-based AMI**, and the public key for SSH access is configured. Terraform also dynamically adjusts the instance count based on the scaling requirements.
- **Security Groups:** The security group is configured to allow inbound traffic on ports 22 (SSH), 5000 (application), 80 (HTTP), and 443 (HTTPS).
- **Key Pair Management:** An AWS EC2 key pair is created for SSH access, allowing secure communication between the instances.
- **Outputs:** After provisioning, Terraform outputs the public IPs of the instances, which are needed for configuring Nginx and managing traffic routing.

❖ **Main.tf :-**

```

provider "aws" {
    region = "us-east-1"
}

terraform {
    required_providers {

```

```

aws = {
  source = "hashicorp/aws"
  version = "5.16.1"
}
}}
resource "aws_key_pair" "key" {
  key_name = "projectkey"
  public_key = file("/home/ubuntu/terraform/projectkey.pub")
}
resource "aws_default_vpc" "default_vpc" { }
resource "aws_security_group" "allow_sshv4" {
  name      = "allow_sshv4"
  description = "Allow SSH and App traffic"
  vpc_id    = aws_default_vpc.default_vpc.id
  ingress {
    description = "Allow SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    description = "Allow App on port 5000"
    from_port   = 5000
    to_port     = 5000
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    description = "Allow HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

```

```

}
ingress {
    description = "Allow HTTPS"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
}
egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
}
tags = {
    Name = "allow_sshv4"
}
}
variable "ami_id" {
    description = "Ubuntu AMI ID"
    default     = "ami-053b0d53c279acc90"
}
variable "instance_count" {
    description = "Number of instances"
    default     = 1
}
resource "aws_instance" "my_ec2" {
    count          = var.instance_count
    ami           = var.ami_id
    instance_type = "t2.micro"
    key_name      = aws_key_pair.key.key_name
    security_groups = [aws_security_group.allow_sshv4.name]
    associate_public_ip_address = true
    tags = { Name = "new-server-${count.index + 1}" }
}

```

```

provisioner "local-exec" {
    command = "echo '${self.public_ip}' >> /home/ubuntu/terraform/ip_save.txt"
}
output "public_ips" {
    value = aws_instance.my_ec2[*].public_ip
}
output "latest_instance_ip" {
    value = element(aws_instance.my_ec2[*].public_ip, length(aws_instance.my_ec2) - 1)
}

```

5. Ansible for Configuration Management

Ansible is used to automate the configuration of the newly provisioned EC2 instances. The **Ansible Playbook** installs the necessary software, including **Docker**, **NGINX**, and the **Flask Application**:

- **Docker:** The application is Dockerized using a Python-based Docker image, and the latest image is pulled from Docker Hub for deployment. Docker containers are configured to run the Django application on port 5000.
- **NGINX:** Nginx is installed and configured to act as a reverse proxy, forwarding traffic to the active Docker container. The upstream configuration is dynamically updated to include the newly provisioned instance IP, and Nginx is reloaded to apply the changes.

❖ **Configuration_instance.yml :-**

```
- name: Install Docker, Deploy App, and Configure NGINX with stub_status
```

```
hosts: servers
```

```
become: yes
```

```
tasks:
```

```
- name: Install required system packages
```

```
apt:
```

```
name:
```

```
- apt-transport-https
```

```
- ca-certificates
```

```
- curl
```

```
- software-properties-common
```

```
state: present
```

```
update_cache: yes
```

```
- name: Add Docker GPG key
```

```
apt_key:
```



```
url: https://download.docker.com/linux/ubuntu/gpg
state: present
- name: Add Docker APT repository
  apt_repository:
    repo: deb [arch=amd64] https://download.docker.com/linux/ubuntu focal stable
    state: present
- name: Install Docker
  apt:
    name: docker-ce
    state: present
    update_cache: yes
- name: Ensure Docker service is started and enabled
  service:
    name: docker
    state: started
    enabled: yes
- name: Pull Docker image
  docker_image:
    name: pratiklanjewar/mydemoapp
    tag: latest
    source: pull
- name: Run Docker container
  docker_container:
    name: main_app_container
    image: pratiklanjewar/mydemoapp:latest
    state: started
    restart_policy: always
    ports:
      - "5000:5000"
- name: Install Nginx
  apt:
    name: nginx
    state: present
    update_cache: yes
```

6. Nginx Reverse Proxy and Load Balancing

Nginx plays a critical role in managing traffic across dynamically scaled EC2 instances. It functions both as a reverse proxy and a load balancer, ensuring that the Dockerized Django application can handle varying user loads efficiently and without downtime.

• Reverse Proxy Functionality

Nginx is configured to function as a reverse proxy, meaning it listens for incoming client requests and forwards them to the appropriate backend EC2 instance where the Dockerized Django application is running. This approach hides the internal architecture of the server network from users while centralizing the entry point for the application.

- All user traffic (typically on port 80) is directed to the main server, where Nginx is running.
- Depending on the active or available EC2 instances, Nginx proxies requests to one or more backend instances, where each instance is running the Django app inside a Docker container on port 5000.

This setup decouples client requests from direct server access, enhancing security and improving scalability by managing backend servers transparently.

❖ /etc/nginx/nginx.conf :-

```
http {  
  
    include /etc/nginx/upstreams/new_ips.conf;  
    server {  
        listen 80;  
        location / {  
            proxy_pass http://backend_servers;  
            proxy_set_header Host $host;  
            proxy_set_header X-Real-IP $remote_addr;  
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        }  
        location /nginx_status {  
            stub_status;  
            allow 127.0.0.1;  
            deny all;  
        }  
    }  
}
```

• Dynamic IP Management via Script Automation

The dynamic nature of your infrastructure means that EC2 instances can be created or destroyed based on CPU usage and traffic demand. To support this flexibility, Nginx's configuration must be updated automatically to reflect the current set of active backend servers.

- A Bash script continuously monitors the system load and active Nginx connections.
- When the need for a new server is detected, Terraform provisions a new EC2 instance, and Ansible configures it with Docker and the Django application.
- The script captures the public IP address of the newly created instance and uses the sed command to update the new_ips.conf file, which contains the IPs of the backend servers used in the upstream block of the Nginx config.

❖ **/etc/nginx/upstream/new_ips.conf :-**

```
upstream backend {
    server 18.204.122.15:5000;
    server 3.83.56.121:5000;
    #new instance is created the ip is added in it ...
}
```

- **Traffic Switching and Load Distribution**

As traffic increases and new EC2 instances are added, Nginx automatically starts distributing load among all available backend servers listed in the updated upstream block.

- Nginx uses round-robin load balancing by default, which evenly distributes incoming requests among the listed servers.
- This guarantees that no single instance is overwhelmed while others remain idle, improving performance and resource utilization.
- If needed, more advanced load-balancing methods like least connections or IP-hash can be configured, depending on future scalability needs.

7. Automated Scaling and Continuous Monitoring

The scaling system operates continuously in the background, with the script running in a loop. The script checks resource utilization at regular intervals and decides whether scaling is necessary.

- **Scaling Criteria:** New instances are launched if the CPU usage exceeds 80% and the number of active connections surpasses 300.
- **New Instance Provisioning:** If scaling is triggered, the system provisions a new EC2 instance via Terraform, configures it with Docker and Nginx using Ansible, and updates the Nginx configuration to distribute traffic to the new instance.

5.4 Output

1) Created the main server and configured

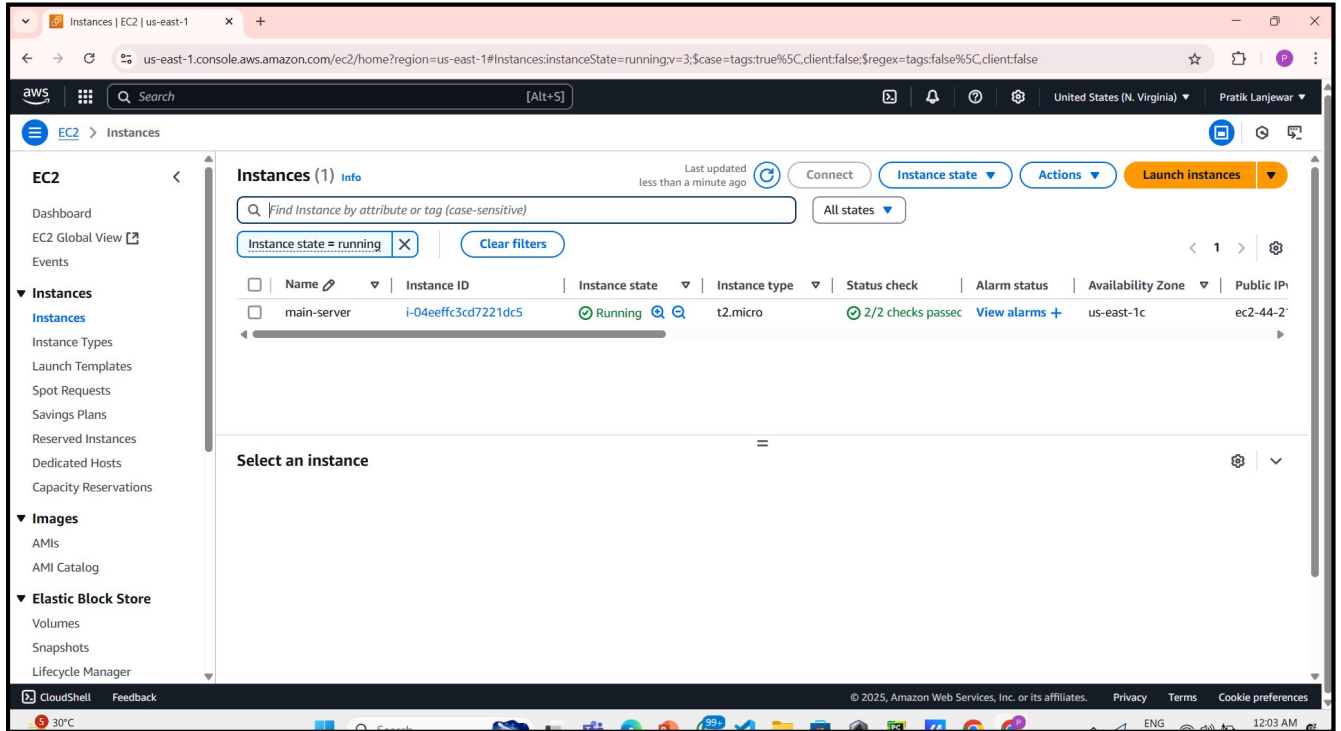


Fig. Screenshot showing AWS EC2 Console Showing Main Server Instance Running and Configured

2) First Release to main server Environment

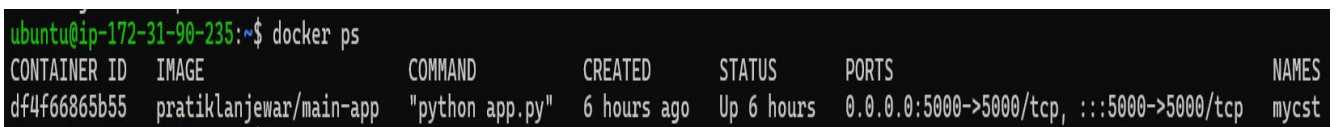


Fig. Docker Container Running on Main Server (docker ps Output)

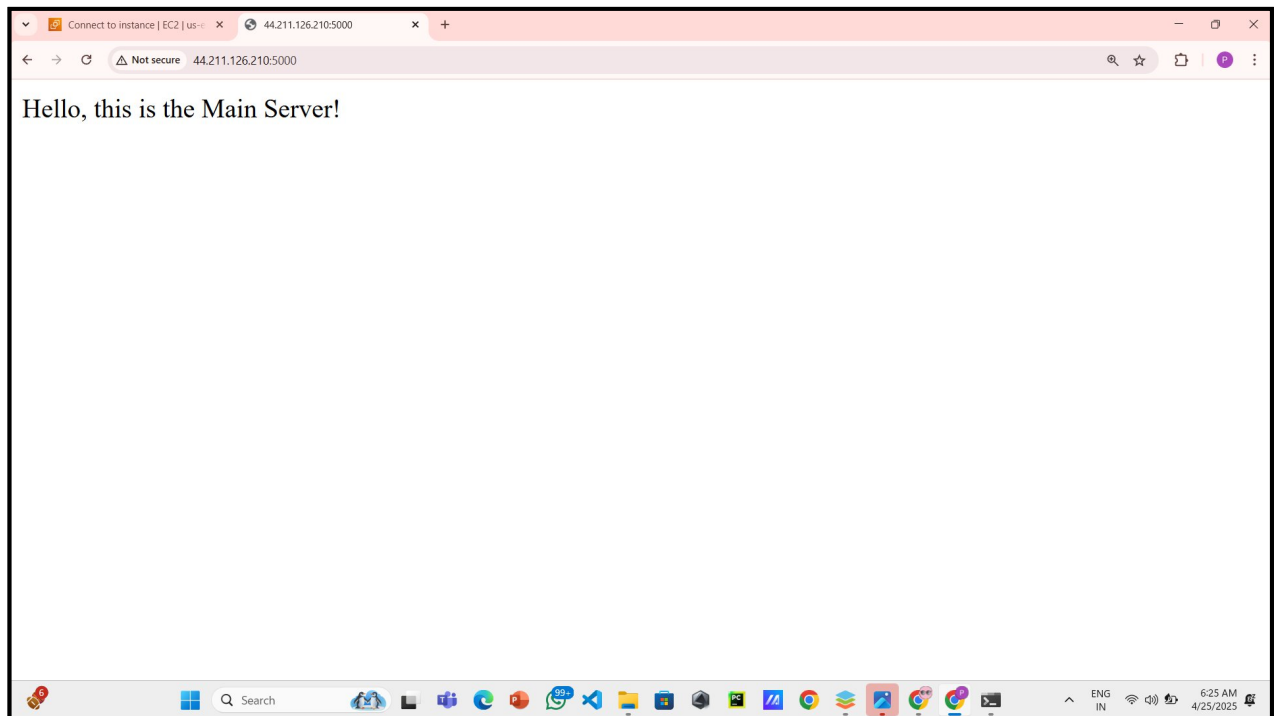


Fig. Web Application Accessible on Port 5000 – Initial Deployment Verification

3) Nginx Configuration on main server

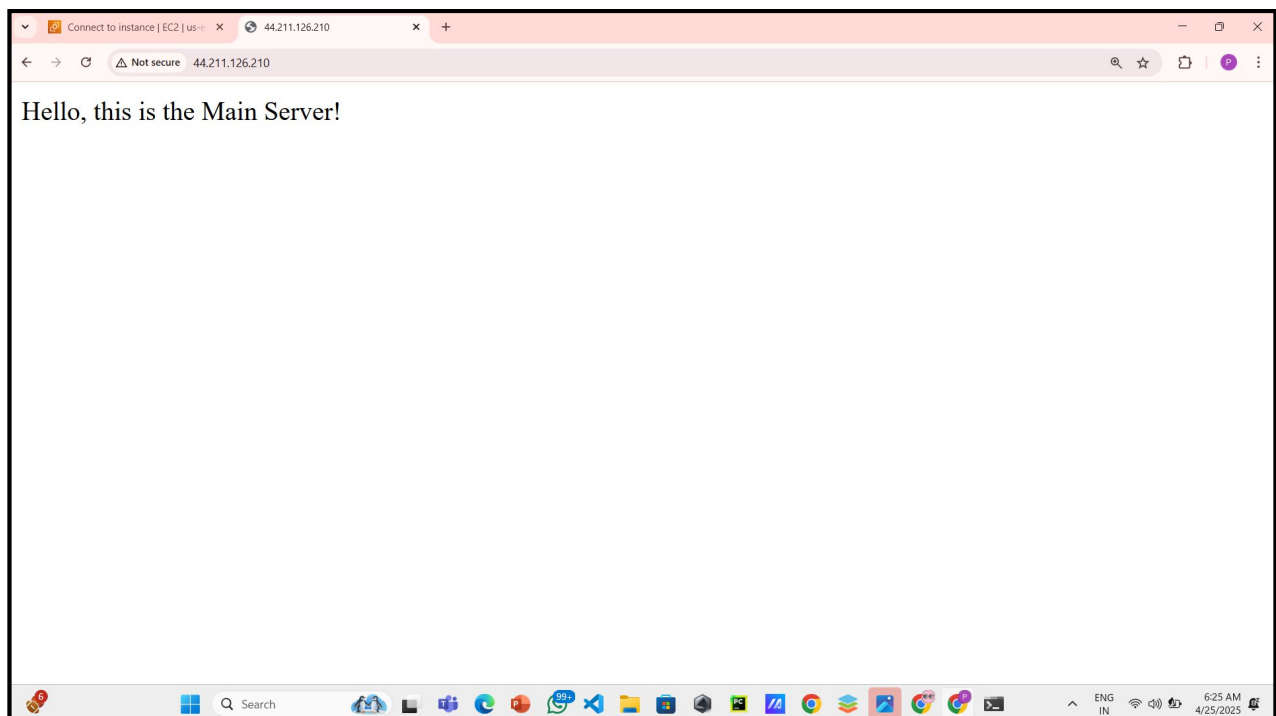


Fig. Web Application Accessible via NGINX Reverse Proxy on Main Server

3) Monitoring the load on your web application, including CPU utilization, active connections

```
ubuntu@ip-172-31-90-235:~/terraform$ ./increment_instance.sh
CPU Usage: 1%
Active Connections: 1
Required Instances: 1
Current Instances: 2
CPU Usage: 7.41%
Active Connections: 1
Required Instances: 1
Current Instances: 2
CPU Usage: 3.85%
Active Connections: 1
Required Instances: 1
Current Instances: 2
CPU Usage: 3.88%
Active Connections: 1
Required Instances: 1
Current Instances: 2
```

Fig. Monitoring Script (increment_instance.sh) Running to Track CPU Utilization and Active Connections

4) After the load apply on the web application

```
ubuntu@ip-172-31-90-235:~$ wrk -t50 -c300 -d100s http://44.211.126.210/
Running 2m test @ http://44.211.126.210/
50 threads and 300 connections
|
```

Fig. Load Testing with wrk – 300 Active Connections, 50 Threads for 2 Minutes

```
CPU Usage: 5.66%
Active Connections: 1
Required Instances: 1
Current Instances: 1
CPU Usage: 100%
Active Connections: 301
Required Instances: 2
Current Instances: 1
High load detected. Scaling up...
aws_key_pair.key: Refreshing state... [id=projectkey]
aws_default_vpc.default_vpc: Refreshing state... [id=vpc-0a9c03ea58a2da72e]
aws_security_group.allow_sshv4: Refreshing state... [id=sg-0f1e1e6df93339d2a]
aws_instance.my_ec2[0]: Refreshing state... [id=i-03c6617670f164f32]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.my_ec2[0] will be created
```

Fig.High Load Detected – Triggering Auto-Scaling Based on Metrics

```

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  ~ latest_instance_ip = "3.82.138.31" -> (known after apply)
  ~ public_ips         = [
    ~ "3.82.138.31" -> (known after apply),
  ]
aws_instance.my_ec2[0]: Creating...
aws_instance.my_ec2[0]: Still creating... [10s elapsed]
aws_instance.my_ec2[0]: Provisioning with 'local-exec'...
aws_instance.my_ec2[0] (local-exec): Executing: ["/bin/sh" "-c" "echo '54.209.152.159' >> /home/ubuntu/terraform/ip_save.txt"]
aws_instance.my_ec2[0]: Creation complete after 12s [id=i-0a837cef67795555b]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

latest_instance_ip = "54.209.152.159"
public_ips = [
  "54.209.152.159",
]
New instance IP: 54.209.152.159
Waiting for SSH on 54.209.152.159...
.....

```

Fig.Terraform Output – New EC2 Instance (New Server 1) Provisioned Successfully

```

PLAY [Install Docker, Deploy App, and Configure NGINX with stub_status] *****

TASK [Gathering Facts] *****
ok: [server_1]

TASK [Install required system packages] *****
changed: [server_1]

TASK [Add Docker GPG key] *****
changed: [server_1]

TASK [Add Docker APT repository] *****
changed: [server_1]

TASK [Install Docker] *****
changed: [server_1]

TASK [Ensure Docker service is started and enabled] *****
ok: [server_1]

TASK [Pull Docker image] *****
changed: [server_1]

TASK [Run Docker container] *****
changed: [server_1]

TASK [Install Nginx] *****
changed: [server_1]

PLAY RECAP *****
server_1 : ok=9  changed=7  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

```

Fig.Ansible Playbook Successfully Configured New Server 1

```

nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

```

Fig. NGINX Configured on New Server 1 – Ready for Load Balancing

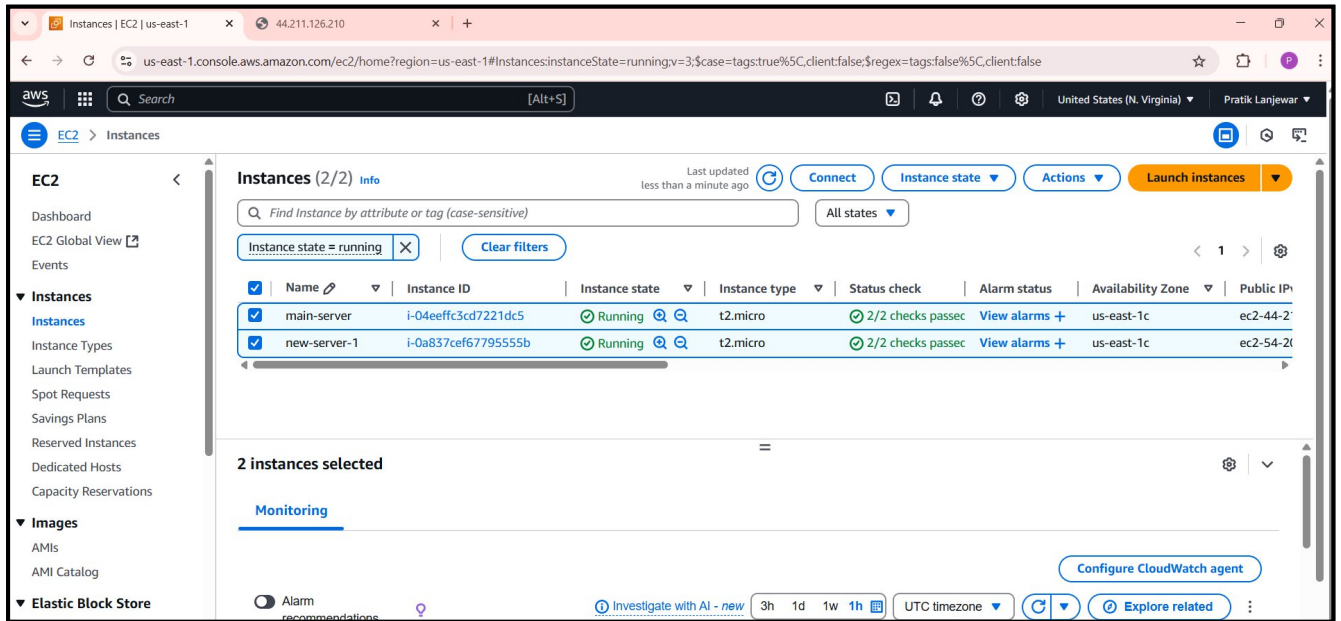


Fig. AWS EC2 Console Showing Two Running Instances – Main Server and New Server 1

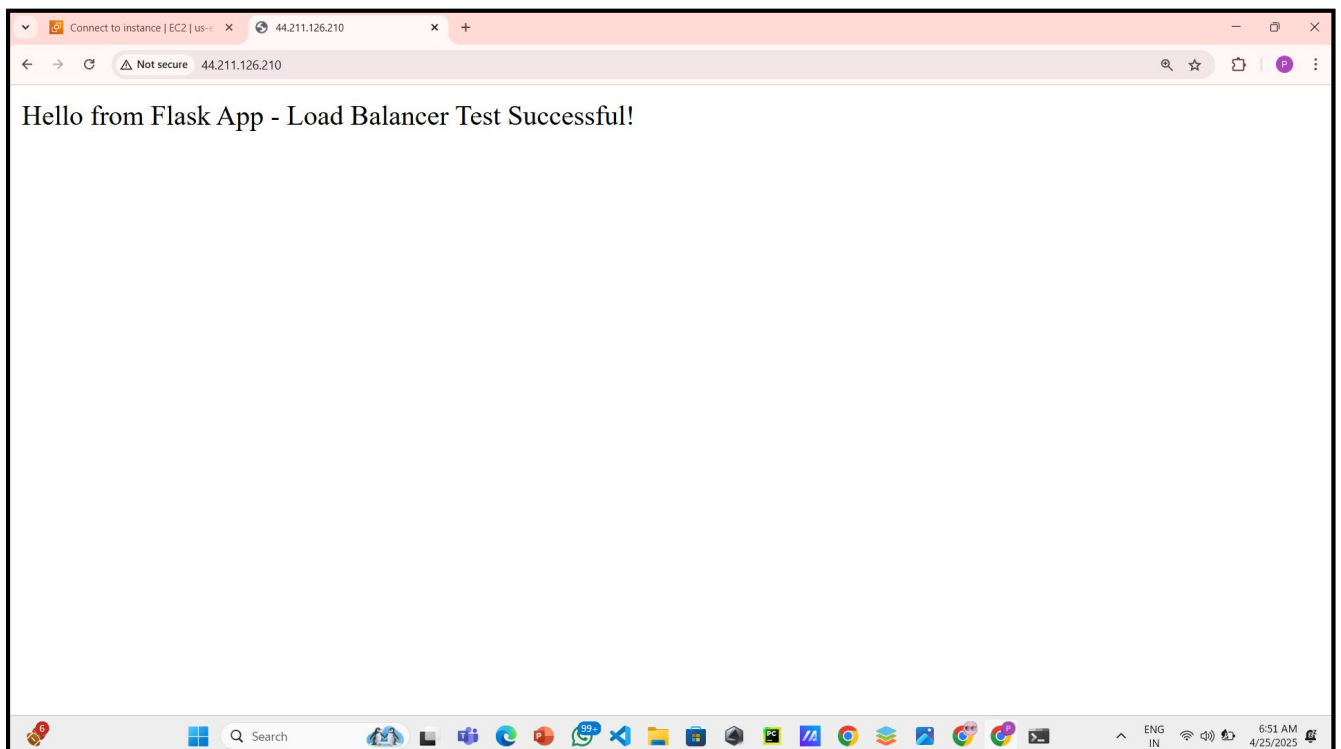


Fig. Web Application Load-Balanced – New Server 1 Accessible via Main Server IP

5.5 Challenges Faced and Overcoming Them:

- **Frequent Changes in EC2 IPs**

Challenge: Dynamic IP allocation made traffic routing inconsistent.

Solution: Used aws ec2 describe-instances in a shell script to fetch IPs and update Nginx config dynamically.

- **Manual Server Configuration**

Challenge: Setting up Docker, Flask app, and Nginx on new instances was error-prone.

Solution: Automated configuration with Ansible after Terraform provisions each instance.

- **Traffic Routing via Nginx**

Challenge: Manual updates to Nginx caused misconfigurations.

Solution: A script updated the Nginx upstream block using sed and reloaded the service automatically.

- **Scaling Based on Load**

Challenge: Monitoring and scaling manually was inefficient.

Solution: A monitoring script triggered Terraform and Ansible based on CPU and traffic thresholds.

- **Zero Downtime During Scaling**

Challenge: Scaling risked brief service interruptions.

Solution: Existing servers stayed active; new ones were added only after full configuration.

- **Configuration Consistency**

Challenge: Ensuring uniform setup across instances.

Solution: Centralized scripts and playbooks maintained consistency across the environment.

5.6 New Skills and Knowledge Gained:

Through the Automated Server Scaling and Load Management project, I gained valuable hands-on experience in the following key areas:

- **Infrastructure as Code (IaC) using Terraform**

Learned to automate the provisioning of AWS EC2 instances, enabling dynamic scaling based on system demand.

- **Server Configuration Management using Ansible**

Acquired skills in automating the setup of Docker, Flask applications, and Nginx on newly provisioned instances, ensuring consistency and efficiency.

- **Containerization with Docker**

Built and deployed Docker containers for a Python Flask application, gaining knowledge in container lifecycle, image optimization, and application portability.

- **Dynamic Load Balancing using Nginx**

Gained practical experience in configuring Nginx as a reverse proxy and load balancer with automated IP updates for backend EC2 instances.

- **Shell Scripting for Infrastructure Automation**

Developed custom shell scripts to monitor traffic load, trigger scaling operations, and automate Nginx configuration updates.

- **Cloud Automation via AWS CLI**

Used AWS CLI to programmatically manage EC2 instances, retrieve instance metadata, and integrate IP data into the deployment pipeline.

- **Monitoring and Scaling Logic**

Designed a logic-based system that scales infrastructure based on CPU load and active connection count, simulating real-time traffic-driven autoscaling

CHAPTER 6

ACHIEVEMENTS AND CONTRIBUTIONS

With Informatrix IT Solutions Pvt. Ltd., I experienced various real-world challenges, working both independently and collaboratively. This internship helped me build resilience and confidence, preparing me for future opportunities in the corporate IT world. The project I worked on, Automated Server Scaling and Load Management, allowed me to apply innovative solutions and technical skills to create a meaningful, scalable infrastructure. Below are some highlights of my learning and achievements.

6.1 Significant Accomplishments:

- **Exploring DevOps Concepts:**

During the internship, I was introduced to key DevOps practices, including automation, CI/CD pipelines, infrastructure provisioning, and dynamic scaling. I gained hands-on experience with tools such as Terraform, Ansible, Jenkins, AWS CLI, and Docker, understanding how development and operations can merge to deliver reliable and scalable systems.

- **Implemented an Automated Server Scaling Project:**

One of my major accomplishments was contributing to the Automated Server Scaling and Load Management project. It involved real-time traffic-based provisioning of EC2 instances using Terraform, along with automated Docker and Nginx setup using Ansible. This taught me how enterprise-level applications can be scaled dynamically while ensuring high availability.

- **Developed Monitoring and Load Management Logic:**

I designed and implemented shell scripts to monitor CPU usage and active connections in real-time. This monitoring system served as a trigger for dynamic scaling, adding or removing EC2 instances to meet traffic demand — a vital skill in infrastructure automation.

- **Enhanced Technical and Soft Skills:**

I improved my proficiency in infrastructure scripting, containerization, system architecture, and cloud management. I also enhanced my communication, collaboration, and project ownership through regular team interactions, feedback sessions, and task tracking.

6.2 Contributions and Organizational Benefits:

- **Engaged in Real-Time Infrastructure Projects:**

My direct involvement in the project led to the successful deployment of an auto-scaling infrastructure using Terraform and Jenkins. By managing both deployment automation and load balancing, I ensured minimal downtime and optimized resource usage.

- **Cross-Team Collaboration:**

I collaborated with developers, system administrators, and project leads to integrate automation scripts, resolve deployment issues, and enhance Nginx traffic routing. This collaborative approach ensured alignment with project goals and improved overall team efficiency.

- **Resource Optimization through Automation:**

By creating a scalable, automated infrastructure, I helped reduce manual configuration efforts and system downtime. The use of Ansible for provisioning and Docker for application deployment made server setups faster and more reliable, reducing operational overhead and increasing deployment speed.

- **Streamlined Continuous Deployment:**

I integrated Jenkins pipelines with GitHub for automated deployment. Each code push triggered provisioning, configuration, and load balancing updates—streamlining DevOps practices and improving delivery timelines.

6.3 Positive Feedback and Recognition:

- **Academic Recognition:**

My faculty mentor appreciated my technical execution and problem-solving abilities in the Automated Server Scaling project. I was acknowledged for effectively translating theory into a fully functional system with zero-downtime deployment capabilities.

- **Company Appreciation:**

At Informatix IT Solutions Pvt. Ltd., my supervisors recognized my initiative and deep understanding of DevOps principles. My ability to use Terraform, Ansible, and Jenkins to automate critical infrastructure components was highly appreciated.

- **Project Recognition:**

The Automated Server Scaling project was highlighted for its scalability, efficiency, and minimal downtime during traffic spikes. Senior leadership noted the value of its modular architecture, its future-readiness, and the effective use of real-time metrics for decision-making. My solution-driven approach added strategic value to the company's cloud infrastructure initiatives.

CHAPTER 7

LEARNING OUTCOMES

7.1 Learning Experience:

- **Refining My Expertise in DevOps and Cloud Automation:**

During my internship and while working on the Automated Server Scaling and Load Management project, I significantly strengthened my technical expertise. I gained hands-on experience with tools like Terraform, Ansible, Docker, and Jenkins, and learned how to design and implement infrastructure that scales based on traffic demands. This project deepened my understanding of cloud automation, continuous deployment, and load balancing in real-world environments, which are vital skills for building reliable and scalable cloud-native systems.

- **Embracing Continuous Improvement:**

One of the biggest takeaways from this project was the importance of continuous learning and iterative enhancement. From optimizing Nginx configurations to improving shell scripts that monitored system performance, I constantly refined the system for better stability and responsiveness. This mindset of iterative problem-solving and performance tuning will remain valuable throughout my career.

- **Strengthened Problem-Solving Abilities:**

The project presented several challenges—managing dynamic IPs, automating server configuration, and ensuring zero downtime during scaling. Each issue demanded a structured approach, from writing monitoring scripts to automating EC2 provisioning. These problem-solving experiences enhanced my analytical thinking, scripting capabilities, and ability to apply infrastructure-as-code principles in real-time scenarios.

7.2 Relating Experiences to Academic Coursework:

- **Bridging Theory and Practice:**

The project aligned well with concepts I studied in my coursework, such as networking, operating systems, and software architecture. I applied theoretical knowledge—like client-server models and load balancing algorithms—in a live cloud environment, giving me practical insight into how academic principles translate into production-level systems.

- **Learning Beyond the Classroom:**

I also developed skills that extend beyond my curriculum, particularly in infrastructure automation and DevOps. Tools like Terraform, Ansible, and AWS CLI were not part of my academic syllabus, but learning and applying them enriched my technical toolkit and prepared me for the modern demands of cloud-based software engineering.

7.3 Impact on Career Goals:

- **Industry Exposure and Networking:**

This internship introduced me to real-world DevOps environments and provided valuable interaction with experienced engineers and mentors. These connections broadened my industry perspective and offered guidance on career pathways in cloud computing, automation engineering, and infrastructure design.

- **Boost in Career Confidence and Direction:**

Successfully completing the Automated Server Scaling project gave me clarity about my career goals. I now feel more confident pursuing roles like DevOps Engineer, Cloud Infrastructure Engineer, or Site Reliability Engineer. The skills I acquired—from scripting and configuration management to cloud orchestration—will be instrumental in my journey toward becoming a full-stack DevOps professional.

- **Preparedness for Real-World Problem Solving:**

Working on a dynamic infrastructure project taught me how to approach complex technical challenges under pressure. Whether it was automating scaling decisions or troubleshooting deployment issues, the experience strengthened my ability to solve problems efficiently, a key requirement for any role in the tech industry.

- **Stronger Resume and Job Readiness:**

This project added a substantial achievement to my resume, showcasing my ability to build, automate, and manage scalable cloud infrastructure. It has made me more competitive in the job market and ready for interviews in DevOps and Cloud roles, where hands-on project experience is highly valued.

CHAPTER 8

CHALLENGES AND LESSON LEARNED

8.1 Encountered Challenges and Solutions:

- **Secure Handling of Dynamic Infrastructure**

Challenge: As new EC2 instances are automatically created during scaling, managing access credentials and secure communications becomes complex.

Improvement: Implement automated SSH key rotation and use AWS Systems Manager (SSM) for secure instance access instead of relying solely on key pairs.

- **Monitoring Deployment Triggers and Server Health**

Challenge: The current scaling mechanism relies on basic CPU thresholds, which may not reflect real user load or application performance.

Improvement: Integrate Prometheus for detailed resource monitoring and Grafana dashboards to visualize trends. This enables smarter, behavior-based scaling and accurate rollback decisions based on real-time system metrics.

- **Terraform State Management and Consistency**

Challenge: When scaling happens multiple times, Terraform state files can become inconsistent, especially in team environments.

Improvement: Use remote state management with AWS S3 and state locking with DynamoDB to maintain accurate, team-safe infrastructure records during scaling operations.

- **Nginx Configuration Scalability**

Challenge: The Nginx upstream block is updated manually with IPs via a script, which may not scale well with high-frequency scaling events.

Improvement: Move towards service discovery-based Nginx updates using tools like Consul or integrate with AWS Application Load Balancer (ALB) for more dynamic backend updates.

8.2 Areas for Improvement in Project:

- **Security Hardening**

Improve overall infrastructure security by restricting Jenkins access, securely managing SSH keys, and regularly reviewing Nginx configuration to mitigate vulnerabilities.

- **Monitoring and Rollback Validation**

Integrate monitoring tools like Prometheus and Grafana to track system performance and health. This would support better decision-making and ensure automatic rollbacks are triggered based on real-time insights.

- **Infrastructure as Code (IaC)**

Expand usage of Terraform or AWS CloudFormation to handle all infrastructure setup, ensuring consistency, improving maintainability, and reducing human error in deployment processes.

8.3 Lessons Learned:

- **Hard Work Leads to Results**

Persistently working through technical challenges demonstrated the value of commitment and focus, especially in managing infrastructure at scale. Every issue resolved added depth to my DevOps knowledge.

- **Foundation is Everything**

A reliable base architecture, particularly in a load-balanced environment, proved essential. The project's success depended on the seamless integration of compute, networking, and automation layers.

- **Aim Higher After Each Milestone**

Every stage of the project provided new learning curves—from infrastructure design to automation—which has prepared me to take on more advanced DevOps and Cloud Engineering roles in the future.

- **Adaptability and Continuous Learning**

The constantly evolving project scope and toolchain reinforced the importance of staying adaptable. Quickly learning and implementing new technologies became a critical part of project success.

CHAPTER 9

RECOMMENDATIONS

9.1 Feedback for Organization:

- **Supportive Environment:**

During my internship, I experienced a welcoming and respectful environment. Senior team members and mentors were approachable and always ready to help. There was no sense of hierarchy or discrimination, which made it easy to ask questions and learn freely.

- **Project Involvement:**

I was given hands-on responsibility in a real-time project, which helped me develop both technical and interpersonal skills. Going forward, giving interns opportunities to work on multiple projects—either individually or in teams—can help build leadership, teamwork, and communication skills more effectively.

9.2 Areas of Improvement:

- **Better Task Planning:**

At times, managing tasks across the different layers of the Automated Server Scaling project (like Terraform, Jenkins, Ansible, and NGINX) became difficult. Improving time allocation and project planning can help interns handle responsibilities more smoothly.

- **Post-Internship Job Support:**

It would be helpful if the organization offered guidance or support for job placement after the internship. Even providing opportunities within the company could boost the intern's confidence and allow them to continue contributing with already-built experience and understanding of the company culture.

9.3 Recommendations for Future Interns:

- **Go Beyond the Assigned Role:**

While focusing on your core project is important, try exploring related technologies and tasks. Learning about areas outside your main assignment—like cloud security, monitoring, or advanced automation—will set you apart and broaden your skillset.

- **Build Professional Connections:**

Don't hesitate to talk to mentors, teammates, or HR. Sharing your strengths and interests may open up more suitable or challenging tasks for you. Networking with professionals and staying visible in the organization helps shape your future opportunities.

- **Focus on Communication Skills:**

Technical knowledge is important, but how you express and present your work matters even more. Make an effort to be clear, confident, and professional when interacting with clients or presenting solutions—it makes a lasting impression and can lead to more responsibilities.

- **Be Open to Learning from Challenges:**

Every obstacle, whether it's a script error or a failed deployment, is a chance to grow. Tackling real-time issues in the scaling system taught me valuable lessons in resilience, patience, and creativity. Embrace these moments as stepping stones to success.

CHAPTER 10

CONCLUSION

10.1 Summary:

- **Professional Experience:**

My internship experience at Informatrix IT Solutions Pvt. Ltd. was both enriching and transformative. It provided a platform to sharpen my technical skills and apply them meaningfully across real-time projects, especially in the domain of DevOps and cloud automation.

- **Project Exposure:**

Working on the Automated Server Scaling project offered valuable insights into modern deployment strategies and scalable infrastructure design. The project was executed smoothly, with all functional and technical requirements successfully met. It holds strong potential for future enhancements and real-world applications.

- **Soft Skills Development:**

The internship not only improved my technical expertise but also helped me grow in terms of communication, teamwork, and presentation skills. Regular project discussions and collaborative sessions significantly enhanced my ability to articulate ideas and work efficiently in a professional setting.

- **Additional Learning:**

The exposure to DevOps practices enhanced my analytical thinking and problem-solving abilities. Hands-on experience with tools like Jenkins, Docker, Ansible, and Terraform deepened my understanding of CI/CD workflows and made the development process more effective and engaging.

10.2 Gratitude:

I am deeply grateful for the opportunity to intern with Informatrix IT Solutions Pvt. Ltd., a company that fosters innovation and learning. The supportive and collaborative environment played a significant role in my personal and professional growth, allowing me to develop skills that will benefit my future career. I would also like to extend heartfelt thanks to my professors for their unwavering support and guidance throughout this journey. Their mentorship and continuous feedback were instrumental in shaping both my individual learning and the successful completion of my project.

In conclusion, this internship has been a cornerstone in my career development—an experience that I will always value. It has inspired me to aim higher and pursue excellence in every step of my professional journey.